Project 1 — SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint

Title: SupportDesk AI Chat And Auto Triage Any Customer Complaint

Type: Multi-agent (2 agents) · **Stack:** Python, raw OpenAI SDK (no agent framework, on purpose) · **Level:** Beginner

Tagline: A 2-agent support desk you run in your terminal — a Concierge agent that just chats, and a Triage agent that turns a free-text complaint into a clean, structured ticket.

> This is the **reading guide** — everything you need to build the project yourself: the story, the folder structure and why each part exists, setup, the 4 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/`.

Charter (what this project is)

A terminal app with two agents working together: Concierge (talks to the user) and Triage (turns a complaint into structured data). A router sends each message to the right one. No agent framework yet — just the raw OpenAI SDK. This project proves two things with real code: (1) you can call an LLM correctly, and (2) you understand what "multi-agent" really means, at its simplest. It also proves a third, quieter thing: you can lay out a Python project the way a real team would.

Jump to: Read first · Project structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4 · Checklist · Publish

The Story — what you're actually building

Imagine your company gets support messages all day — some people just want to chat or ask a quick question, others are genuinely upset about something broken and need a proper ticket filed. Right now a person reads every single message and decides which is which by hand. That's slow, and it doesn't scale.

You're building a small terminal app that does both jobs for you. One part, the **Concierge**, is a normal chat agent — it holds a real conversation, remembers what was said earlier, and replies

naturally. The other part, the **Triage** agent, has a narrower, stricter job: read a messy, free-text complaint and turn it into a clean record a support system could actually use — who it's from, what kind of problem it is, how urgent it is, and a one-line summary. A small **router** sits in front of both and decides, automatically, which one should handle each message — so nobody has to say "this one's a complaint" out loud.

Each Step below builds one piece of that, in order: Step 1 proves you can talk to the model at all. Step 2 turns that into a real, remembering conversation. Step 3 adds the Triage agent and the error handling that keeps the app from crashing. Step 4 adds the router that ties Concierge and Triage together — the actual "multi-agent" part.

> **Before you read further — think about it yourself:** if you were building this by hand, what's the very first thing you'd need to prove works, before writing any chat logic at all? And once you have two different jobs (chatting, and filing a ticket), how would your code decide, on its own, which one a given message needs?

What you're actually building, in one line: a terminal app where a router looks at each message and sends it to one of two agents — a chat agent, or a data-extraction agent.

Why this needs to exist: a real support inbox mixes casual questions with real complaints, and paying a person to sort every message by hand does not scale.

When you'd reach for this at a real job: when a product gets more than one kind of incoming message, and nobody wants to force the user to pick a category — a support inbox, a multi-purpose chatbot, any intake form.

How it works, mechanically: the router asks the model one small, focused question ("concierge or triage?") and gets back a checked answer. Then the matching agent function runs. The two agents never talk to each other — the router just picks one per message.

Why not one big prompt that does both? One prompt trying to do open-ended chat *and* strict structured output tends to do both worse — the model gets confused about which mode it's in. **Why not make the user type `/chat` or `/extract`?** That pushes the sorting work onto the user; the whole point is that the system figures it out.

Where This Fits in the 5-Project Arc

Every project in this curriculum is a real multi-agent system. What changes is *how many* agents there are, *how* they're built, and *how* they work together:

```
Project 1 (you are here) → 2 agents, Python router, raw SDK
                           (Concierge + Triage)
Project 2                → 2 agents, hand-built agent loop, tools
```

```
                          (Worker + Verifier)
Project 3                 → 3 agents, LangGraph, RAG + human approval
                          (Retriever + Reasoner + Approval)
Project 4                 → 4-5 agents, one Supervisor directs the rest
                          (Supervisor + Research/Analysis/Writer/Reviewer)
Project 5                 → Project 4, made production-ready
                          (API, database, Docker, testing)
```

Project 1's 2 agents are kept simple on purpose (no tools, no framework) so the only new multi-agent idea is "a router sends work to the right agent."

What to Read First (docs this project uses)

Everything this project needs was taught in Docs 01-04. Use this table when a step says "Read first", or whenever an idea feels unfamiliar:

You need to understand…	Read this	Used in
venv, `pip`, `requirements.txt`	Doc01 — Virtual environments	Setup
`.env`, never committing secrets	Doc01 — `.env` files	Setup, Step 1
`load_config()`, `MissingConfigError`	Doc01 — Errors + Doc01 Build Task	Step 1
`get_logger()`, why not `print()`	Doc01 — Logging	Step 1 onward
The client and the message list	Doc04 — The client	Step 1
Prompts in `.txt` files: `load_prompt()`, `.read()`	Doc04 — Basic exercise, Approach 2	Step 2 onward
Why the model forgets everything	Doc03 — No memory	Step 2
Tokens and the "4 characters" rule	Doc03 — Tokens	Step 2
The context window limit	Doc03 — Context window	Step 2
Streaming a reply	Doc04 — Streaming	Step 2
Pydantic schemas, `Literal`, `X | None`	Doc04 — Structured output	Steps 3, 4
"Valid JSON" vs. "the right shape"	Doc02 — JSON	Step 3
Which errors to retry, which not	Doc04 — Error types + Doc02 — Retrying	Step 3
The whole app, as a practice run	Doc04 Build Task	All steps

You need to understand…	Read this	Used in
Routed vs. supervisor patterns	Doc11 — Patterns (look ahead — just the Sequential and Supervisor rows)	Step 4

`config.py`, `exceptions.py` and `logger.py` in this project are **Doc01's Build Task files, unchanged** — only their location and import lines change (see below). `send_message()` and `stream_message()` are **Doc04's Build Task functions**, with the small changes each step points out.

Project Structure (the professional layout)

A general AI-project template has many folders: `api/`, `rag/`, `chains/`, `notebooks/`, `docker/`, and more. A professional doesn't copy all of them — they keep **only the folders this project has a real job for**, and add the rest the day a real need appears. Below is the right-sized layout for this project, what each part is for, and what was left out on purpose.

Names: the repo folder vs. the Python package

- **Repo folder:** `SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/` — already created for you. This is the exact name you give the GitHub repo, so the folder on your machine and the repo online always match.
- **Python package:** `src/`, right next to `main.py`. It holds all the app's code, and every file imports from it the same way: `from src.config import load_config`.
- **Files:** `snake_case.py`, named for the one job they do (`history.py`, not `utils2.py`). Test files start with `test_`, so anyone can spot them at a glance.
- **Prompt files:** named after the agent that uses them (`triage.txt` belongs to `agents/triage.py`).

The final layout (after Step 4)

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├── data/
│   └── sample/
│       └── messages.json       test complaints + routing cases
├── scripts/
│   └── triage_samples.py       runs Triage on every sample message
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── concierge.py    concierge_agent() -> str
│   │   ├── router.py       decide() -> RouteDecision
│   │   └── triage.py       triage_agent() -> SupportTicket | None
```

```
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py          create_client(), send/stream_message()
│   ├── prompts/
│   │   ├── concierge.txt   the Concierge's system prompt
│   │   ├── router.txt      the router's system prompt
│   │   └── triage.txt      the Triage agent's system prompt
│   ├── services/
│   │   ├── __init__.py
│   │   └── desk_service.py handle_message(): router -> agent
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── history.py      estimate_tokens(), trim_history()
│   │   ├── logger.py       get_logger()                (Doc01)
│   │   └── prompts.py      load_prompt(name)
│   ├── __init__.py
│   ├── config.py           Config, load_config()       (Doc01)
│   ├── exceptions.py       MissingConfigError          (Doc01)
│   └── schemas.py          SupportTicket, RouteDecision
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py real API calls
│   └── unit/
│       ├── test_config.py      no API key needed
│       ├── test_history.py     no API key needed
│       └── test_schemas.py     no API key needed
├── .env                    your real key — never committed
├── .env.example            the setting names, no real values
├── .gitignore              keeps .env, .venv/, __pycache__/ out of git
├── LICENSE                 lets others reuse your code (MIT)
├── main.py                 the terminal loop — the file you run
├── README.md               the repo's front page (copied in at the end)
└── requirements.txt        the 3 libraries, for pip install -r
```

Why each folder exists

- **`data/sample/`** — the sample messages, as JSON. **Why:** the same inputs every time, shared by the script and the tests, instead of messages retyped by hand in 3 places. (Doc02 taught reading JSON.)
- **`scripts/`** — small tools you run by hand, which are not part of the app. **Why:** `triage_samples.py` prints tickets for a *person* to judge (e.g. after rewording `triage.txt`). That's a tool, not a test and not a feature.
- **`src/`** — the package itself: every other `.py` file the app is made of. **Why:** one clear name to import from, everywhere — `main.py`, tests and scripts all say `from src... import ...`.

- **`agents/`** — one file per agent (router, concierge, triage). **Why:** each agent has its own prompt and its own job; separate files make "these are 2 separate agents" something you can see, not just claim.
- **`models/`** — the one place that talks to the OpenAI SDK (`llm.py`). **Why:** if you change model, or later swap in LangChain (Doc05), you change one file, not every agent. ("Models" here means *language models*, not Pydantic models — those live in `schemas.py`.)
- **`prompts/`** — the system prompts, as plain `.txt` files. **Why:** you can reword how an agent behaves without touching Python, and a reviewer can read every prompt in one folder. A prompt change shows up in git as its own clean diff.
- **`services/`** — the "business flow": take a message, ask the router, call the right agent. **Why:** `main.py` only reads the keyboard and prints. The flow itself lives here, so Project 5 can call the same `handle_message()` from a web API with no terminal code.
- **`utils/`** — small helpers that aren't about one agent: logging, history trimming, prompt loading. **Why:** shared by several files; putting them in one place stops copy-paste.
- **`tests/integration/`** — tests that call the real API. **Why:** kept separate because they cost money, need a key, and are slower — you run them on purpose, not by accident.
- **`tests/unit/`** — tests that run with no API key, no network, in under a second. **Why:** you can run them after every change, and a CI server could too (Doc12-13).
- **`main.py`** (a file, at the root) — the one file you run: `python main.py`. **Why:** it sits right next to the `src/` folder, so Python finds the package there.

Why each root file exists

- `.env.example` — tells a teammate which settings to create, without any real value.
- `.gitignore` — stops secrets (`.env`), the venv and Python's cache files from being pushed.
- `LICENSE` — the MIT license text, created in Setup. Without one, nobody is legally allowed to reuse your code.
- `README.md` — what someone sees first on GitHub: what it is, how to run it.
- `requirements.txt` — the library list for a plain `pip install -r requirements.txt` (the Doc01 habit).

Why each file exists (what and why)

Every file you'll write, in one place — the same What/Why list the documents give after each tree. The steps below build these files one at a time; come back here whenever you forget why a file exists.

- `data/sample/messages.json`
  - **What:** A JSON object with two lists: `"triage"` (the sample complaints) and `"routing"` (messages with the agent each should go to).
  - **Why:** The script and the tests share one copy of the test inputs, so they always use the same data.

- `scripts/triage_samples.py`
  - **What:** One `main()` that loads the config, reads `messages.json` with `json.loads()`, runs `triage_agent()` on every sample complaint, and prints each ticket.
  - **Why:** After you change a prompt, you read the tickets yourself. A test can check the format; only a person can judge if a ticket is good.
- `agents/concierge.py`
  - **What:** Two functions: `start_history()` returns a list with the concierge system prompt, and `concierge_agent(client, history, message)` calls `trim_history()` and then `stream_message()`.
  - **Why:** It answers normal questions and remembers the conversation, while keeping the history under the model's size limit.
- `agents/router.py`
  - **What:** One function, `decide(client, message)`. It sends the message with the router prompt to `.parse(..., response_format=RouteDecision)`, starts from a safe default (Concierge), uses the parsed answer if there is one, logs the agent and the reason, and returns the `RouteDecision`.
  - **Why:** Without it the user would have to type a command to pick the agent. The logged reason lets you explain every routing choice later.
- `agents/triage.py`
  - **What:** One function, `triage_agent(client, message)`. It sends the triage prompt and the message to `.parse(..., response_format=SupportTicket)`, returns `None` on a `ValidationError` or a refusal, and otherwise returns the parsed `SupportTicket`.
  - **Why:** Complaints become clean, structured tickets. Returning `None` on a bad answer means no half-valid ticket ever reaches the rest of the app.
- `models/llm.py`
  - **What:** A `MODEL` constant and four functions: `create_client(config)` returns `OpenAI(api_key=...)`; `check_api_key_at_startup(client)` sends a 1-token request and returns `False` on `AuthenticationError`; `send_message()` and `stream_message()` add the user turn to the history, call the model, add the reply and return it.
  - **Why:** It is the only file that talks to the OpenAI SDK, so a model change or an SDK change touches one file. Without the startup check, a wrong key would only fail in the middle of the first chat.
- `prompts/*.txt`
  - **What:** Plain text files, one per agent: the router's sorting rule, the concierge's tone, and triage's rules (such as "never guess a name").
  - **Why:** You can read and change each agent's behavior in one place, without touching the code.
- `services/desk_service.py`

  - **What:** Two functions. `print_ticket(ticket)` (written in Step 3) prints the four ticket fields, showing "(not given)" for a missing name. `handle_message(client, history, message)` (added in Step 4) calls `decide()`, then either `triage_agent()` + `print_ticket()` or `concierge_agent()`.
  - **Why:** The router → agent flow lives outside `main.py`, which only reads input. A web API could later call `handle_message()` without any terminal code.
- `utils/history.py`
  - **What:** Two functions: `estimate_tokens(history)` adds up the length of every message and divides by 4; `trim_history(history, max_tokens)` deletes `history[1]` (the oldest turn, never the system prompt) in a `while` loop until the estimate fits.
  - **Why:** The model has a size limit. Without trimming, a long chat would eventually fail with a 400 error.
- `utils/logger.py`
  - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
  - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
  - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
  - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `__init__.py` (every code folder)
  - **What:** An empty file. You create one in every folder that holds Python code — nothing goes inside it.
  - **Why:** Python only treats a folder as a package when this file is there. Without it, an import like `from <package>.<folder> import ...` fails with `ModuleNotFoundError`.
- `config.py`
  - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
  - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.

    - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception`, and real bugs would be hidden along with the expected problem.
- `schemas.py`
    - **What:** Two Pydantic classes: `SupportTicket` (`customer_name: str | None`, `issue_category` and `urgency` as `Literal[...]`, `summary`) and `RouteDecision` (`agent: Literal["concierge", "triage"]`, `reason`).
    - **Why:** The model's answers come back as checked objects. Without them you'd pull fields out of free text by hand, and a made-up name or a misspelled category would slip through.
- `tests/integration/test_live_agents.py`
    - **What:** Test functions that call the real API, added step by step: a first reply, the key check with a fake key, a 3-turn memory test, the sample tickets, no invented name, and the routing cases. Each prints `OK` or `WRONG`.
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `tests/unit/test_config.py`
    - **What:** Three test functions, no API call, each printing `OK` or `WRONG`: `require_env()` raises `MissingConfigError` for a missing and for an empty setting (a made-up setting name, so your `.env` is never touched), and `load_config()` returns a `Config` when a key is set.
    - **Why:** It proves a missing key stops the app with one clear error, with no API call — the first thing that must work.
- `tests/unit/test_history.py`
    - **What:** Three small `test_...()` functions that build a history by hand, call `estimate_tokens()` or `trim_history()`, and print `OK` or `WRONG`: 40 characters count as 10 tokens, a short chat is not trimmed, and a long chat keeps the system prompt and the newest turn.
    - **Why:** It proves the system prompt and the newest message are never deleted — in a second, with no API key and no cost.
- `tests/unit/test_schemas.py`
    - **What:** Small test functions that create `SupportTicket` and `RouteDecision` objects by hand: one valid (a ticket with no name), and some invalid ones inside `try` / `except ValidationError`. Each prints `OK` or `WRONG`.
    - **Why:** If someone loosens a schema by mistake, this test catches it before the model starts inventing names or agent names.
- `main.py`
    - **What:** One function, `main()`. It loads the config, builds the client, checks the key, creates the chat history, then loops: read a line, stop on `/quit`, call `handle_message()` inside `try`, and turn each OpenAI error (bad key, rate limit, too long) into one clear message.

- **Why:** The app needs one starting point that owns the keyboard and the error messages. All the real work is in other files, so this one stays short.

How imports work in this layout

- Every folder with Python files gets an empty `__init__.py`. It marks the folder as a package, so `src.agents.router` is importable.
- Python looks for imports in the folder you **run from**. `main.py` and `src/` sit side by side in the repo folder, so `python main.py` finds `from src.utils.logger import get_logger` right there.
- Tests and scripts live in sub-folders, so run them as modules from the repo folder: `python -m tests.unit.test_history`. The `-m` tells Python to start from the folder you're in — the repo folder — so `src` is found there too.
- Always run commands from the **repo folder**. The prompt loader, the tests and the script also use paths like `data/sample/messages.json` that start there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
`api/`, `routes.py`	This is a terminal app, no web server	Project 5 (Doc12)
`rag/`, `vectorstore.py`	No documents to search	Project 3 (Doc08)
`chains/`	No LangChain yet — raw SDK on purpose	Doc05, Project 3
`configs/*.yaml`	2 settings fit in `.env`; YAML needs a new library	When settings grow
`notebooks/`	Nothing to explore or plot	Only if you do experiments
`Dockerfile`, `docker-compose.yml`	Nothing to deploy yet	Project 5 (Doc12)
`Makefile`	3 short commands, all listed in the README	Optional, any time
`.github/workflows/`	CI is taught later	Doc12-13
`tests/conftest.py`	No pytest yet — plain test scripts, like Doc04	Doc13
`docs/`	One README is enough for this size	Bigger projects

How the structure grows, step by step

Step	New files	Changed files
Setup	root files, `src/__init__.py`	—
1	`config.py`, `exceptions.py`, `utils/logger.py`, `models/llm.py`, `main.py`, `tests/unit/test_config.py`, `tests/integration/test_live_agents.py`	—
2	`utils/history.py`, `utils/prompts.py`, `prompts/concierge.txt`, `tests/unit/test_history.py`	`models/llm.py`, `main.py`, `test_live_agents.py`
3	`schemas.py`, `agents/triage.py`, `prompts/triage.txt`, `services/desk_service.py`, `data/sample/messages.json`, `scripts/triage_samples.py`, `tests/unit/test_schemas.py`	`main.py`, `test_live_agents.py`
4	`agents/router.py`, `agents/concierge.py`, `prompts/router.txt`	`services/desk_service.py`, `schemas.py`, `main.py`, `messages.json`, both test files

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`) that's:

```
cd project_1_supportdesk_chat_and_triage
cd SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Create `.gitignore` **before** your first commit:

```
# secrets — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
# Python's compiled cache files
```

```
__pycache__/
*.pyc
```

3. Create `.env` (your real key) and `.env.example` (same names, no values):

```
# .env — never commit this file
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

```
# .env.example — commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

4. Create `requirements.txt`:

```
openai>=1.40
pydantic>=2.0
python-dotenv>=1.0
```

`>=` means "this version or newer" — the oldest versions this code is known to work with.

5. Create the package folder with an empty `src/__init__.py`, then install the libraries:

```
pip install -r requirements.txt
```

Check it worked: from the repo folder, `python -c "import src; print('ok')"` prints `ok`. It works because the `src/` folder sits right there — that's why every command runs from the repo folder.

Optional — real test complaints: for realistic messy input for Step 3, you can pull a few rows from the CFPB Consumer Complaint Database — a free, public dataset of real complaints. Not required — the 3 inputs below work fine.

6. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
```

```
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

A Real Example (so this isn't just theory)

Scenario: the user describes a problem in plain text. Your code turns it into a `SupportTicket`:

```
customer_name: str | None
issue_category: Literal["billing", "technical", "account_access", "other"]
urgency: Literal["low", "medium", "high"]
summary: str          # one sentence describing the issue
```

Test these 3 inputs (Step 3 saves them in `data/sample/messages.json`, so you reuse the same ones every time):

1. `"Hi, I'm Sarah Khan. I was charged twice this month. Please fix it today."` → `issue_category="billing"`, `urgency="high"`, `customer_name="Sarah Khan"`.
2. `"can't log into my account, tried resetting password twice, still nothing"` → `issue_category="account_access"`, `customer_name=None` (no name was given — this checks your code doesn't make one up).
3. `"just wondering what your refund policy is, no rush"` → `issue_category` could be `"other"` or `"billing"` — a judgment call; decide, and be ready to explain. `urgency="low"`.

Why input 2 matters: a missing piece of information must stay empty (`None`) instead of the model inventing a name. This is a real problem you'll actually hit.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** most "I called an LLM" demos don't hold a real conversation, don't produce checked structured data, and don't have more than one role. This one does all three — with no framework, in a real package layout.
- **Why it matters:** it shows you understand what happens underneath the tools you'll use later.
- **When you'd build something like this at a job:** any small internal tool or first version of a product. A lot of real features are exactly this: a small router in front of two or three well-defined model calls.
- **How it's built:** you type a message → the router picks Concierge or Triage → Concierge remembers and streams, Triage extracts once. Every likely failure is handled on purpose.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The conversation gets too long for the model	Estimate tokens before each call and drop the oldest turns first (Step 2's `trim_history()`)
The structured-output call doesn't match your schema	Catch Pydantic's `ValidationError` by name and tell the user — never pass bad data on
You find out the API key is wrong mid-chat	Test the key with one 1-token call at startup (Step 1)
The router picks the wrong agent	Every decision is logged with a reason (Step 4) — read the log, then reword `prompts/router.txt`
`ModuleNotFoundError: No module named 'src'`	You're not in the repo folder, or you ran a test as `python tests/...` instead of `python -m tests....`

Built During These Documents

01_python_foundations → 02_apis_http_json → 03_llm_fundamentals → 04_openai_api

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks yourself, before you open your editor.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 answers one message, Step 2 is a chat app, Step 3 adds tickets, Step 4 is the full 2-agent desk. Each step only adds or changes a few files — nothing is thrown away. Each step says what to build, which files it adds, why they exist,

and how to check it works. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command in the steps runs from the **repo root** (`SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/`) with the venv active.

How testing works in this project

Story: you check SupportDesk the way a support lead checks a new team member — a small check on each skill as soon as it's learned, not one big exam at the end. Every step adds the tests for the feature it adds. So when something breaks, the newest `WRONG` points straight at the newest code. **If not:** every check would be you typing into the app by hand, and a change in Step 4 could quietly break Step 2's memory without anyone noticing.

- **Two kinds of test:**
  - `tests/unit/` — code that needs no model: settings, history trimming, schemas. No API key, no cost, under a second. **When:** after every change to the file it tests — run them all the time.
  - `tests/integration/test_live_agents.py` — the parts that need the real model: replies, memory, tickets, routing. Real API calls (a fraction of a cent). **When:** at the end of every step, and after changing a prompt, `MODEL` or `models/llm.py`.
- **One pattern for every case** — so the output never needs judging:

```
def test_something() -> None:
    result = "OK"
    if <it came out wrong>:
        result = "WRONG"
    print(f"case 1 (what it checks) -> {result}")
```

- **An error you expect** (like `MissingConfigError`): start with `result = "WRONG"`, and set `result = "OK"` inside `except TheError:`.
- **One function per case**, all called at the bottom under `if __name__ == "__main__":`. Each file numbers its own cases, and case numbers never change once written.
- **Model answers vary**, so check one fact (the reply contains `4`, the ticket is not `None`), never the exact wording.
- **Run from the repo root with `-m`:** `python -m tests.unit.test_config` (dots, no `.py`). Running the file by its path fails with `No module named 'src'`, because Python then starts looking for imports in `tests/unit/`.
- **Got a `WRONG`?** Read that one test function, then the code it calls. Fix the code — change the test only if the test itself checks the wrong thing.

Step 1 — A Script That Sends One Message and Prints the Reply

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 1 of 4

- **Where you are now:** Setup is done — the repo folder, venv, `.env`, `requirements.txt`, and an empty `src/` package. Nothing runs yet.
- **What's new in this step:** Doc01's config/logging files, `models/llm.py` (the only file that talks to OpenAI), a `main.py` that sends one message, and the first tests.
- **Why:** before any chat or agents, prove the foundations work together — settings load, logging works, the key is valid, one call comes back. If they don't, you find out now, alone, not mixed into Step 3's logic.
- **How:** `load_config()` → `get_logger()` → `create_client()` → `check_api_key_at_startup()` (a 1-token call) → `send_message()` → print.
- **Working after this step:** `python main.py` prints the model's answer to "What's 2+2?", and both test files pass.

When you'll hit this for real: "prove the connection works" is the first step on *any* new AI project, before any feature.

Read first: Doc01 Build Task (config + logging), Doc04 — The client and the list of messages.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Bring in Doc01's `exceptions.py`, `config.py` and `logging_setup.py` as `src/exceptions.py`, `src/config.py` and `src/utils/logger.py`. Change only the import lines (`from src.exceptions import MissingConfigError`) — nothing else.
2. In `models/llm.py`, write `create_client(config)`, `check_api_key_at_startup(client)` (one 1-token call; `False` on `AuthenticationError`) and Doc04's `send_message(client, history, user_input)`.
3. In `main.py`, write `main()`: load config, get a logger, build the client, check the key, send `"What's 2+2?"`, print the reply.
4. Run `python main.py`. Then put a fake key in `.env` and check you get one clear line, not a traceback.
5. Write `tests/unit/test_config.py` (Doc01's three config checks, each printing `OK` or `WRONG`) and `tests/integration/test_live_agents.py` with cases 1-2 (a real reply; a bad key caught at startup). Run both with `python -m`.

Your files after Step 1:

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├── src/
│   ├── models/
│   │   ├── __init__.py  (new)
│   │   └── llm.py       (new) create_client(), check_api_key_at_startup(),
```

```
│   │                          send_message()
│   ├── utils/
│   │   ├── __init__.py  (new)
│   │   └── logger.py    (new) get_logger()
│   ├── __init__.py
│   ├── config.py        (new) Config, require_env(), load_config()
│   └── exceptions.py    (new) MissingConfigError
├── tests/
│   ├── integration/test_live_agents.py  (new) cases 1-2, real API
│   └── unit/test_config.py              (new) no API key needed
├── .env                (Setup)
├── .env.example        (Setup)
├── .gitignore          (Setup)
├── LICENSE             (Setup)
├── main.py             (new) sends one message, prints the reply
└── requirements.txt    (Setup)
```

Why these files (what and why):

- `models/llm.py`
  - **What:** A `MODEL` constant and three functions: `create_client(config)` returns `OpenAI(api_key=...)`; `check_api_key_at_startup(client)` sends a 1-token request and returns `False` on `AuthenticationError`; `send_message()` adds the user turn to the history, calls the model, adds the reply and returns it.
  - **Why:** It is the only file that talks to the OpenAI SDK, so a model change or an SDK change touches one file. Without the startup check, a wrong key would only fail in the middle of the first chat.
- `utils/logger.py`
  - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
  - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `config.py`
  - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
  - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
  - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.

    - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception`, and real bugs would be hidden along with the expected problem.
- `tests/integration/test_live_agents.py`
    - **What:** Two test functions that call the real API, each printing `OK` or `WRONG`: `test_first_reply()` checks the reply to "What's 2+2?" contains 4, and `test_bad_key()` checks that `check_api_key_at_startup()` rejects a fake key.
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `tests/unit/test_config.py`
    - **What:** Three test functions, no API call, each printing `OK` or `WRONG`: `require_env()` raises `MissingConfigError` for a missing and for an empty setting (a made-up setting name, so your `.env` is never touched), and `load_config()` returns a `Config` when a key is set.
    - **Why:** It proves a missing key stops the app with one clear error, with no API call — the first thing that must work.
- `main.py`
    - **What:** One function, `main()`. It loads the config (and stops with one clear line on `MissingConfigError`), gets a logger, builds the client, checks the key, sends "What's 2+2?" with `send_message()`, and prints the reply.
    - **Why:** The app needs one starting point that owns the keyboard and the error messages. All the real work is in other files, so this one stays short.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 1 (missing setting)** — `tests/unit/test_config.py`
    - **Check:** with a made-up setting name removed from `os.environ`, `require_env()` raises `MissingConfigError`.
    - **Why:** a missing key must stop the app at startup with one clear line.
    - **When:** after any change to `config.py` or `exceptions.py`.
    - **If not:** a missing key would surface minutes later as a confusing `AuthenticationError` from OpenAI.
- **case 2 (empty setting)** — `tests/unit/test_config.py`
    - **Check:** the same name set to `""` also raises `MissingConfigError`.
    - **Why:** `OPENAI_API_KEY=` with nothing after it is "set" but never usable.
    - **When:** same as case 1.
    - **If not:** the app would start happily and fail on the first real call.
- **case 3 (config loads)** — `tests/unit/test_config.py`
    - **Check:** with `OPENAI_API_KEY` set in `os.environ`, `load_config()` returns a `Config` holding that key.

    - **Why:** cases 1–2 only prove the *error* path; this proves the values actually come back.
    - **When:** same as case 1.
    - **If not:** a missing `return value` in `require_env()` passes cases 1–2, and every setting silently becomes `None`.
- **case 1 (first reply)** — `tests/integration/test_live_agents.py`
    - **Check:** `send_message()` with "What's 2+2?" returns a reply that contains `4`.
    - **Why:** proves the key, the model name and `send_message()` work together, end to end.
    - **When:** at the end of the step, and after changing `MODEL` or `models/llm.py`.
    - **If not:** a typo in the model name would be found by the first user, not by you.
- **case 2 (bad key)** — `tests/integration/test_live_agents.py`
    - **Check:** `check_api_key_at_startup()` returns `False` for `OpenAI(api_key="sk-fake-key-123")`.
    - **Why:** the startup check is what turns a bad key into one clear line instead of a traceback mid-chat.
    - **When:** same as case 1.
    - **If not:** if the check stopped catching `AuthenticationError`, a bad key would crash the app on the user's first message.

Step 2 — A Real Terminal Chat With Memory and Live Streaming

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 2 of 4

- **Where you are now:** after Step 1 the app answers **one** fixed message and exits. It has settings, logging, a key check and `send_message()`.
- **What's new in this step:** a chat loop, a growing `history` list (the memory), `stream_message()`, a system prompt in `prompts/concierge.txt`, and `utils/history.py` to trim long chats. Tests: `test_history.py`, and a memory case in `test_live_agents.py`.
- **What stays the same:** `config.py`, `exceptions.py`, `utils/logger.py`, `create_client()`, `check_api_key_at_startup()`, `send_message()`.
- **Why:** the model remembers nothing between calls (Doc03), so the app must keep and resend the conversation itself. Streaming makes replies appear as they're written. Trimming stops a long chat from going over the context limit.
- **How:** each turn: read input → `trim_history()` → `stream_message()`, which appends both the user turn and the reply to `history`.
- **Working after this step:** a real terminal chat that remembers what you said earlier, and all tests pass.

When you'll hit this for real: every chat feature you'll ever build starts with exactly this — a growing message list plus streaming.

Read first: Doc03 — No memory, Doc03 — Tokens, Doc04 — Streaming.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add Doc04's `stream_message(client, history, user_input)` to `models/llm.py` — with `flush=True` on the print, so each piece appears the moment it arrives.
2. Write `prompts/concierge.txt` (the assistant's personality) and `utils/prompts.py` with `load_prompt(name)`.
3. Write `utils/history.py`: `estimate_tokens(history)` (characters ÷ 4, Doc03's rule) and `trim_history(history, max_tokens)` — drops the oldest turn, never the system prompt.
4. Change `main.py` into a loop: read input, `/quit` to exit, trim, stream the reply.
5. Write `tests/unit/test_history.py` and run it: `python -m tests.unit.test_history`. Add case 3 (memory) to `tests/integration/test_live_agents.py`.
6. Test memory by hand: say your name, chat about something else, then ask "what's my name?".

Your files after Step 2:

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├── src/
│   ├── models/llm.py      (changed) + stream_message()
│   ├── prompts/
│   │   └── concierge.txt  (new) the assistant's system prompt
│   ├── utils/
│   │   ├── history.py     (new) estimate_tokens(), trim_history()
│   │   ├── logger.py
│   │   └── prompts.py     (new) load_prompt(name)
│   ├── config.py
│   └── exceptions.py
├── tests/
│   ├── integration/test_live_agents.py  (changed) + case 3, memory
│   └── unit/test_history.py             (new) runs with no API key
├── .env                (Setup)
├── .env.example        (Setup)
├── .gitignore          (Setup)
├── LICENSE             (Setup)
├── main.py             (changed) a chat loop with memory
└── requirements.txt    (Setup)
```

Why these files (what and why):

- `models/llm.py`
  - **What:** A `MODEL` constant and four functions: `create_client(config)` returns `OpenAI(api_key=...)`; `check_api_key_at_startup(client)` sends a 1-token request and returns `False` on `AuthenticationError`; `send_message()` and

`stream_message()` add the user turn to the history, call the model, add the reply and return it.
    - **Why:** It is the only file that talks to the OpenAI SDK, so a model change or an SDK change touches one file. Without the startup check, a wrong key would only fail in the middle of the first chat.
- `prompts/concierge.txt`
    - **What:** This text file holds the concierge prompt. The code loads it with `load_prompt("concierge")`.
    - **Why:** You can read and change each agent's behavior in one place, without touching the code.
- `utils/history.py`
    - **What:** Two functions: `estimate_tokens(history)` adds up the length of every message and divides by 4; `trim_history(history, max_tokens)` deletes `history[1]` (the oldest turn, never the system prompt) in a `while` loop until the estimate fits.
    - **Why:** The model has a size limit. Without trimming, a long chat would eventually fail with a 400 error.
- `utils/prompts.py`
    - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
    - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `tests/integration/test_live_agents.py`
    - **What:** Step 1's two cases plus `test_memory()`: three turns with `send_message()`, and `OK` only if turn 3's reply contains the order number from turn 1.
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `tests/unit/test_history.py`
    - **What:** Three small `test_...()` functions that build a history by hand, call `estimate_tokens()` or `trim_history()`, and print `OK` or `WRONG`: 40 characters count as 10 tokens, a short chat is not trimmed, and a long chat keeps the system prompt and the newest turn.
    - **Why:** It proves the system prompt and the newest message are never deleted — in a second, with no API key and no cost.
- `main.py`
    - **What:** One function, `main()`: Step 1's startup, then a loop — read a line, skip empty ones, stop on `/quit`, trim the history with `trim_history()`, and stream the reply with `stream_message()`.
    - **Why:** The app needs one starting point that owns the keyboard and the error messages. All the real work is in other files, so this one stays short.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 1 (40 characters = 10 tokens)** — `tests/unit/test_history.py`
  - **Check:** two messages of 20 characters each give `estimate_tokens()` = 10.
  - **Why:** every trimming decision is based on this number.
  - **When:** after any change to `utils/history.py`.
  - **If not:** a wrong divisor would trim far too much, or never trim at all.
- **case 2 (short chat not trimmed)** — `tests/unit/test_history.py`
  - **Check:** a 2-message history under the budget still has 2 messages after `trim_history()`.
  - **Why:** trimming must only happen when it's needed.
  - **When:** same as case 1.
  - **If not:** the assistant would forget the conversation for no reason.
- **case 3 (long chat trimmed)** — `tests/unit/test_history.py`
  - **Check:** 5 turns of 400 characters with a budget of 250: the system prompt is still first and turn 5 is still last.
  - **Why:** those are the two messages trimming must never delete.
  - **When:** same as case 1.
  - **If not:** a long chat would lose the assistant's personality (the system prompt) or the question just asked.
- **case 3 (memory)** — `tests/integration/test_live_agents.py`
  - **Check:** three turns with one `history` list; the turn-3 reply to "What was my order number?" contains `4471`.
  - **Why:** memory is this step's feature — it proves the whole history is resent every turn.
  - **When:** at the end of the step, and after changing `send_message()`, `stream_message()` or the loop in `main.py`.
  - **If not:** a bug that sends a fresh list each turn would only show up when a user complained.

Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 3 of 4

- **Where you are now:** after Step 2 you have a working chat with memory and streaming — but it can't file a ticket, and one rate limit or too-long message crashes it.
- **What's new in this step:** `schemas.py` (`SupportTicket`), the first agent `agents/triage.py` with `prompts/triage.txt`, a temporary `/extract` command, one `except` per failure in `main.py`, sample data and a script. Tests: `test_schemas.py`, and ticket cases in `test_live_agents.py`.
- **What stays the same:** the chat loop, `models/llm.py`, and everything in `utils/`.
- **Why:** free text has to become clean, checked data that code can trust — and a real app must survive every common failure without losing the conversation.

- **How:** `/extract <text>` → `triage_agent()` → one structured-output call → a `SupportTicket` or `None`. Every OpenAI error is caught in `main.py` and gets the reaction Doc04's error table gives it.
- **Working after this step:** the chat app from Step 2, plus `/extract` to file a ticket, a clear message for every API error, and all tests pass.

When you'll hit this for real: any feature where free text must become clean data — tickets, forms, intake.

Read first: Doc04 — Structured output, Doc04 — Error types, Doc02 — JSON.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `SupportTicket` in `schemas.py`, exactly as in the Real Example. It's the contract between the model's reply and your code.
2. Write `prompts/triage.txt` (strict: never guess a name) and `agents/triage.py` with `triage_agent(client, message) -> SupportTicket | None`. Use `client.beta.chat.completions.parse(...)` like Doc04, check `refusal` first, and return `None` on `ValidationError`.
3. Create `services/desk_service.py` (with an empty `services/__init__.py`) holding `print_ticket(ticket)`, and in `main.py` add `/extract <text>` (a temporary manual switch — Step 4 replaces it) that calls it.
4. Handle the other 3 failures in the loop: bad key (`AuthenticationError` → exit), rate limit (`RateLimitError` → ask the user to wait), too long (`BadRequestError` → drop that message). The 4th, bad structured output, is handled inside `triage_agent()`.
5. Save the 3 inputs in `data/sample/messages.json`, write `scripts/triage_samples.py` and `tests/unit/test_schemas.py`, and add cases 4-5 (tickets, no invented name) to `tests/integration/test_live_agents.py`.
6. Run all three. Check input 2 gives `customer_name=None`.

Your files after Step 3:

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├── data/sample/
│   └── messages.json         (new) the 3 test complaints
├── scripts/
│   └── triage_samples.py     (new) prints a ticket per sample
├── src/
│   ├── agents/
│   │   ├── __init__.py       (new)
│   │   └── triage.py         (new) triage_agent()
│   ├── models/llm.py
│   ├── prompts/
```

```
│   │   ├── concierge.txt
│   │   └── triage.txt        (new)
│   ├── services/
│   │   ├── __init__.py       (new)
│   │   └── desk_service.py   (new) print_ticket()
│   ├── utils/
│   │   ├── history.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── config.py
│   ├── exceptions.py
│   └── schemas.py            (new) SupportTicket
├── tests/
│   ├── integration/
│   │   └── test_live_agents.py (changed) + cases 4-5
│   └── unit/
│       ├── test_history.py
│       └── test_schemas.py   (new) no API key needed
├── .env                (Setup)
├── .env.example        (Setup)
├── .gitignore          (Setup)
├── LICENSE             (Setup)
├── main.py             (changed) + /extract, all error handling
└── requirements.txt    (Setup)
```

Why these files (what and why):

- `data/sample/messages.json`
  - **What:** A JSON object with two lists: `"triage"` (the sample complaints) and `"routing"` (messages with the agent each should go to).
  - **Why:** The script and the tests share one copy of the test inputs, so they always use the same data.
- `scripts/triage_samples.py`
  - **What:** One `main()` that loads the config, reads `messages.json` with `json.loads()`, runs `triage_agent()` on every sample complaint, and prints each ticket.
  - **Why:** After you change a prompt, you read the tickets yourself. A test can check the format; only a person can judge if a ticket is good.
- `agents/triage.py`
  - **What:** One function, `triage_agent(client, message)`. It sends the triage prompt and the message to `.parse(..., response_format=SupportTicket)`, returns `None` on a `ValidationError` or a refusal, and otherwise returns the parsed `SupportTicket`.
  - **Why:** Complaints become clean, structured tickets. Returning `None` on a bad answer means no half-valid ticket ever reaches the rest of the app.
- `prompts/triage.txt`

    - **What:** This text file holds the triage prompt. The code loads it with `load_prompt("triage")`.
    - **Why:** You can read and change each agent's behavior in one place, without touching the code.
- `services/desk_service.py`
    - **What:** One function, `print_ticket(ticket)`. It prints the four ticket fields, one per line, showing "(not given)" when there is no name.
    - **Why:** The router → agent flow lives outside `main.py`, which only reads input. A web API could later call `handle_message()` without any terminal code.
- `schemas.py`
    - **What:** One Pydantic class, `SupportTicket`: `customer_name: str | None`, `issue_category` and `urgency` as `Literal[...]`, and `summary`.
    - **Why:** The model's answers come back as checked objects. Without them you'd pull fields out of free text by hand, and a made-up name or a misspelled category would slip through.
- `test_live_agents.py`
    - **What:** Step 2's cases plus `load_samples()` and two test functions: `test_every_sample_gets_a_ticket()` (each complaint in `messages.json` becomes a ticket) and `test_no_invented_name()` (the complaint with no name gets `customer_name=None`).
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `test_schemas.py`
    - **What:** Two small test functions that build `SupportTicket` objects by hand: one with `customer_name=None` (must be allowed), and one with `urgency="urgent"` (must raise `ValidationError`). Each prints `OK` or `WRONG`.
    - **Why:** If someone loosens a schema by mistake, this test catches it before the model starts inventing names or agent names.
- `main.py`
    - **What:** One function, `main()`: Step 2's loop, plus `/extract <text>`, which calls `triage_agent()` and `print_ticket()`, and one `except` per OpenAI error — bad key (exit), rate limit (ask to wait), too long (drop the message).
    - **Why:** The app needs one starting point that owns the keyboard and the error messages. All the real work is in other files, so this one stays short.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 1 (no name allowed)** — `tests/unit/test_schemas.py`
    - **Check:** `SupportTicket(customer_name=None, ...)` builds without an error.
    - **Why:** "the customer gave no name" must be a real, allowed answer.
    - **When:** after any change to `schemas.py`.

    - **If not:** if someone changed the field to plain `str`, the model would be forced to invent names.
- **case 2 (bad urgency)** — `tests/unit/test_schemas.py`
    - **Check:** `urgency="urgent"` raises `ValidationError`.
    - **Why:** `Literal` limits urgency to `low`, `medium`, `high` — the values the rest of the code compares against.
    - **When:** same as case 1.
    - **If not:** "urgent" would slip through and break any `if urgency == "high"`.
- **case 4 (ticket)** — `tests/integration/test_live_agents.py`
    - **Check:** every message in `data/sample/messages.json` → `triage_agent()` returns a ticket, not `None`.
    - **Why:** the Triage agent is this step's feature.
    - **When:** at the end of the step, and after changing `prompts/triage.txt` or `SupportTicket`.
    - **If not:** a prompt change that breaks extraction would be found by the support team, not by you.
- **case 5 (no name)** — `tests/integration/test_live_agents.py`
    - **Check:** sample 2 (no name in it) gives a ticket with `customer_name=None`.
    - **Why:** this is the project's "no invented data" promise, checked on every run.
    - **When:** same as case 4.
    - **If not:** a reworded prompt could start inventing names, and fake names would reach real tickets. Not a test: `scripts/triage_samples.py` prints every ticket for **you** to read. A test can check that a ticket exists; only a person can judge whether its summary is good.

Step 4 — A Router That Sends Each Message to the Right Agent Automatically (final: 2 agents)

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 4 of 4

Read first: Doc04 — Structured output (the table row "a supervisor picks the next agent"), and, as a look ahead, Doc11 — Patterns (the Sequential and Supervisor rows).

- **Where you are now:** after Step 3 you have a chat (the future Concierge) and a Triage agent — but *you* pick between them by typing `/extract`.
- **What's new in this step:** `RouteDecision` in `schemas.py`, `agents/router.py` with `prompts/router.txt`, `agents/concierge.py` (the chat code moves out of `main.py`), and `services/desk_service.py` for the router → agent flow. `main.py` gets smaller. Tests: an agent-name case in `test_schemas.py`, and routing cases in `test_live_agents.py`.
- **What stays the same:** `agents/triage.py`, `models/llm.py`, `utils/`, and Step 3's error handling (it moves along unchanged).

- **Why:** the program, not the user, should decide who handles each message. That's what turns one script with two modes into a real 2-agent system.
- **How:** each message → `decide()` (a small structured call that returns an agent and a reason, and logs both) → `handle_message()` calls that agent.
- **Working after this step:** the full project — type anything; chat gets a streamed reply, a complaint becomes a ticket — and all tests pass, routing included.

When you'll hit this for real: any system with more than one kind of request, where a person shouldn't have to say which kind it is. Project 4's Supervisor is this same idea, scaled up.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add `RouteDecision` (`agent: Literal["concierge", "triage"]`, `reason: str`) to `schemas.py`.
2. Write `prompts/router.txt` and `agents/router.py` with `decide(client, message) -> RouteDecision`. Log every decision with its reason. If the answer can't be read, fall back to Concierge.
3. Move the chat part into `agents/concierge.py`: `start_history()` and `concierge_agent(client, history, message)`.
4. Add `handle_message()` to `services/desk_service.py` (below Step 3's `print_ticket()`): it asks the router and calls the right agent.
5. Slim `main.py` down: config, key check, then a loop that calls `handle_message()` inside the same error handling. Remove `/extract`.
6. Add routing cases to `messages.json` and the tests. Run everything, then chat with a mix of small talk and complaints — and read the log for the unclear ones.

Your files after Step 4 (final): the full tree in Project Structure. What changed:

```
SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint/
├── data/sample/
│   └── messages.json         (changed) + the "routing" list
├── src/
│   ├── agents/
│   │   ├── concierge.py      (new) start_history(), concierge_agent()
│   │   ├── router.py         (new) decide() -> RouteDecision
│   │   └── triage.py
│   ├── prompts/
│   │   └── router.txt        (new)
│   ├── services/
│   │   └── desk_service.py   (changed) + handle_message()
│   └── schemas.py            (changed) + RouteDecision
├── tests/
│   ├── integration/
```

```
│   │   └── test_live_agents.py (changed) + case 6, routing
│   └── unit/
│       └── test_schemas.py   (changed) + an agent-name case
└── main.py             (changed) only input, key check, error handling
```

Why these files (what and why):

- `data/sample/messages.json`
  - **What:** A JSON object with two lists: `"triage"` (the sample complaints) and `"routing"` (messages with the agent each should go to).
  - **Why:** The script and the tests share one copy of the test inputs, so they always use the same data.
- `agents/concierge.py`
  - **What:** Two functions: `start_history()` returns a list with the concierge system prompt, and `concierge_agent(client, history, message)` calls `trim_history()` and then `stream_message()`.
  - **Why:** It answers normal questions and remembers the conversation, while keeping the history under the model's size limit.
- `agents/router.py`
  - **What:** One function, `decide(client, message)`. It sends the message with the router prompt to `.parse(..., response_format=RouteDecision)`, starts from a safe default (Concierge), uses the parsed answer if there is one, logs the agent and the reason, and returns the `RouteDecision`.
  - **Why:** Without it the user would have to type a command to pick the agent. The logged reason lets you explain every routing choice later.
- `prompts/router.txt`
  - **What:** This text file holds the router prompt. The code loads it with `load_prompt("router")`.
  - **Why:** You can read and change each agent's behavior in one place, without touching the code.
- `services/desk_service.py`
  - **What:** Two functions. `print_ticket(ticket)` (written in Step 3) prints the four ticket fields, showing "(not given)" for a missing name. `handle_message(client, history, message)` (added in Step 4) calls `decide()`, then either `triage_agent()` + `print_ticket()` or `concierge_agent()`.
  - **Why:** The router → agent flow lives outside `main.py`, which only reads input. A web API could later call `handle_message()` without any terminal code.
- `schemas.py`
  - **What:** Two Pydantic classes: `SupportTicket` (`customer_name: str | None`, `issue_category` and `urgency` as `Literal[...]`, `summary`) and `RouteDecision` (`agent: Literal["concierge", "triage"]`, `reason`).

    - **Why:** The model's answers come back as checked objects. Without them you'd pull fields out of free text by hand, and a made-up name or a misspelled category would slip through.
- `test_live_agents.py`
    - **What:** Test functions that call the real API, added step by step: a first reply, the key check with a fake key, a 3-turn memory test, the sample tickets, no invented name, and the routing cases. Each prints `OK` or `WRONG`.
    - **Why:** It proves the parts that need the real model actually work, and each step's new feature gets its own check.
- `test_schemas.py`
    - **What:** Small test functions that create `SupportTicket` and `RouteDecision` objects by hand: one valid (a ticket with no name), and some invalid ones inside `try` / `except ValidationError`. Each prints `OK` or `WRONG`.
    - **Why:** If someone loosens a schema by mistake, this test catches it before the model starts inventing names or agent names.
- `main.py`
    - **What:** One function, `main()`. It loads the config, builds the client, checks the key, creates the chat history, then loops: read a line, stop on `/quit`, call `handle_message()` inside `try`, and turn each OpenAI error (bad key, rate limit, too long) into one clear message.
    - **Why:** The app needs one starting point that owns the keyboard and the error messages. All the real work is in other files, so this one stays short.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 3 (bad agent name)** — `tests/unit/test_schemas.py`
    - **Check:** `RouteDecision(agent="billing", ...)` raises `ValidationError`.
    - **Why:** `handle_message()` only knows two agents, `concierge` and `triage`.
    - **When:** after any change to `schemas.py`.
    - **If not:** if the `Literal` were widened by mistake, the router could pick an agent that doesn't exist.
- **case 6 (routing)** — `tests/integration/test_live_agents.py`
    - **Check:** every `"routing"` sample in `messages.json` → `decide()` picks its `expected` agent.
    - **Why:** routing is this step's feature — clear complaints must become tickets, clear small talk must not.
    - **When:** at the end of the step, and after changing `prompts/router.txt`.
    - **If not:** a small prompt change could quietly send complaints to the Concierge, and no ticket would ever be filed. Not tested on purpose: unclear messages ("what's your refund policy, no rush"). Either agent is a fair answer — read the logged **reason** instead.

Final Deliverable: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — a 2-agent terminal app in a proper package layout. A Concierge holds real multi-turn conversations. A

Triage agent turns any complaint into a checked `SupportTicket`. A router decides which agent handles each message, and logs why.

Why this really is multi-agent (said plainly): a simple **routed** pattern (Doc11). No tool loop yet (Project 2), no shared graph state yet (Project 3). But it is two agents with different jobs and a router choosing between them — the real, simplest definition of multi-agent. Don't describe it as more advanced than that; Project 4 is where routing becomes a real `Command`-based supervisor.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 4 is done, this is how the finished project is run, tested and configured.

Configuration

Settings are read from `.env` (never committed):

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG`, `INFO`, `WARNING` or `ERROR`

The model (`gpt-4o-mini`) is set in one place: `MODEL` in `src/models/llm.py`. Agent prompts are in `src/prompts/`.

Usage

```
python main.py
```

Type any message. Small talk and questions go to the Concierge; problem reports become tickets. Type `/quit` to exit.

To see the Triage agent's ticket for every sample complaint:

```
python -m scripts.triage_samples
```

Running the Tests

Run from the repo root:

```
# unit tests — no API key, no network, under a second
python -m tests.unit.test_config
```

```
python -m tests.unit.test_history
python -m tests.unit.test_schemas

# integration tests — real API calls, needs OPENAI_API_KEY
python -m tests.integration.test_live_agents
```

Every test case prints `OK` or `WRONG`. The unit tests check the config, history trimming and the schemas. The integration tests check a first reply, the startup key check, conversation memory, ticket extraction (including that no customer name is invented), and routing on clear messages.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] `main.py` at the repo root, the package in `src/`; runs with `python main.py`
- [ ] Every folder has one clear job, and you can say why it exists
- [ ] Multi-turn conversation with real memory (Concierge)
- [ ] Doc01's config/logger — no secrets in code, `print()` only for what the user should see
- [ ] Triage tested on all 3 sample inputs, including the missing-name case
- [ ] The router picks the agent automatically — no `/extract` left
- [ ] Every routing decision is logged, with the agent and the reason
- [ ] Survives: bad key, rate limit, too-long message, bad structured output
- [ ] Every test case prints `OK` — `tests/unit/` with no API key, `tests/integration/` with one
- [ ] You wrote each step's tests yourself, and can say for each one what goes wrong without it
- [ ] `.env` is not in git (`git status` doesn't show it)
- [ ] You can explain every line of your own code, without help

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and fill in your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status                      # .env and .venv/ must NOT appear
git add .
git commit -m "SupportDesk AI: 2-agent chat and triage"
```

3. On GitHub, create a new repo named exactly `SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint` (no README — you have one). In the repo's **Description** box, paste the

Short description line from the top of README.md — it's what people see in search results and on your profile. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**